Neural Surrogate Modeling for Snake Robot Locomotion:

Faster Reinforcement Learning via Learned Cosserat Rod Dynamics

Qifan Wen
Electrical and Computer Engineering
The Ohio State University
wen.679@buckeyemail.osu.edu

Advisor: Prof. Abhishek Gupta gupta.706@osu.edu

Abstract

[Placeholder: Write abstract after all experimental results are available.]

Contents

Notation **4**

1 Introduction **4**

2 Related Work **5**

2.1 Neural Surrogates for ODE/PDE 5

2.2 RL Control for Snake and Soft Robots 5

2.3 Physics Backend Comparison 6

3 Neural Network Surrogate Model for Elastica **8**

3.1 Motivation 8

3.2 Elastica Physics 8

3.2.1 Cosserat Rod Formulation 8

3.2.2 Elastica Transition Operator 10

3.2.3 CPG Control 10

3.3 Data Collection 11

3.4 Model Architecture and Training 12

3.4.1 Input Representation 12

3.4.2 Per-Element CPG Phase Encoding . . . 12
3.4.3 Architecture and Prediction . . . 13
3.4.4 Training . . . 13
3.4.5 Hyperparameters . . . 14
3.5 Results . . . 14
3.6 Challenges . . . 16

4 Reinforcement Learning on Elastica 17
4.1 Environment and Training Setup . . . 17
4.2 Direct Simulation Results . . . 17
4.3 Surrogate Environment Results . . . 19
4.4 Challenges . . . 19

5 Reinforcement Learning on DisMech (Choi2025 Follow Task) 19
5.1 Task Description . . . 20
5.2 Reward Design . . . 20
5.3 Experimental Setup . . . 21
5.4 Results . . . 22
5.5 SAC Gradient Instability . . . 23
5.6 Discussion . . . 24

6 Physics-Informed Neural Network 25
6.1 Mathematical Formulation . . . 25
6.1.1 Standard PINN . . . 25
6.1.2 Physics Regularizer . . . 25
6.1.3 DD-PINN Method . . . 26
6.1.4 Limitations of PINNs . . . 26
6.2 PDE System for Snake Robot . . . 27
6.2.1 Differentiable Cosserat Rod Right-Hand Side . . . 27
6.2.2 Physics-Regularized Loss . . . 27
6.2.3 DD-PINN Loss and Training . . . 27
6.2.4 Nondimensionalization . . . 28
6.2.5 DD-PINN Configuration . . . 28
6.3 Results and Comparison . . . 29

7 Conclusion **32**

A Detailed Physics Derivations **32**

A.1 Staggered Grid Discretization 32

A.2 Method of Lines and Explicit Integration 34

A.3 How Spatial Discretization Converts PDEs to ODEs 34

A.3.1 The Staggered Grid Layout 34

A.3.2 The Difference Matrix $\mathbf{D}$ (Node $\rightarrow$ Element) 35

A.3.3 The Transpose $\mathbf{D}^T$ (Element $\rightarrow$ Node) 35

A.3.4 Adjoint Structure and Energy Conservation 36

A.3.5 Stage 1: Eliminating Spatial Derivatives 36

A.3.6 Stage 2: Reduction to First-Order Form 37

A.4 External Force Handling 37

A.5 DisMech Formulation 38

B Issue Tracker **39**

Notation

Symbol	Dimension	Meaning
$\mathbf{s}_t$	$\mathbb{R}^{124}$	Rod state vector at time step t
$\mathbf{a}_t$	$[-1, 1]^5$	Normalized CPG action vector
T	$\mathbb{R}^{124} \times \mathbb{R}^5 \to \mathbb{R}^{124}$	Ground-truth transition operator (PyElastica)
f_θ	$\mathbb{R}^{189} \to \mathbb{R}^{124}$	Neural surrogate model with parameters θ
$\mathbf{z}_t$	$\mathbb{R}^{189}$	Surrogate input (state $\|$ action $\|$ phase encoding)
$\boldsymbol{\phi}_e$	$\mathbb{R}^3$	Per-element CPG phase triplet for element e
$\mathbf{x}(s,t)$	$\mathbb{R}^3$	Rod centerline position at arc length s
$\mathbf{d}(s,t)$	$\mathbb{R}^3$	Material director frame (orientation)
$\mathbf{F}$	$\mathbb{R}^3$	Internal force (shear and tension)
$\mathbf{M}$	$\mathbb{R}^3$	Internal moment (bending and torsion)
κ_e^{target}	$\mathbb{R}$	Target curvature at element e
$\mu_\Delta, \sigma_\Delta$	$\mathbb{R}^{124}$	Training-set mean and std of state deltas
N	21	Number of discretization nodes
N_e	20	Number of discretization elements

1 Introduction

Reinforcement learning for soft and continuum robots is bottlenecked by the cost of first-principles simulation. A snake robot modeled as a Cosserat rod requires resolving elastic body dynamics, anisotropic ground friction, and internally generated muscle moments at every simulation substep — PyElastica (Naughton et al., 2021) uses 500 Position Verlet substeps per control step, costing $\sim$46 ms per environment step. This report presents a neural surrogate $f_\theta \approx T$ that replaces the transition operator with a single GPU forward pass ($\sim$0.001 ms), targeting a four-to-five order-of-magnitude speedup for RL training.

The project has three pillars:

1. **Neural surrogate** — parallel data collection from 16 PyElastica workers, followed by MLP training on state-delta prediction with per-element CPG phase encoding (Section 3).

2. **Direct RL** — PPO on the Elastica backend (Section 4) and SAC/PPO/MM-RKHS on the DisMech backend for the Choi2025 follow task (Section 5).

3. **Physics-informed models** — standard PINN, DD-PINN, and physics-regularized surrogate variants that embed the governing equations into training (Section 6).

Two physics backends are investigated — PyElastica (explicit symplectic) and DisMech (implicit backward Euler) — providing complementary perspectives on surrogate fidelity and RL control.

2 Related Work

2.1 Neural Surrogates for ODE/PDE

Table 1 summarizes neural surrogate methods directly relevant to the Cosserat rod setting.

Table 1: Neural surrogates for soft robot / Cosserat rod dynamics.

Reference	Method	Architecture	Speedup	Accuracy	Key contribution
DD-PINN (Licher et al., 2025b)	Domain-decoupled PINN	Feedforward NN	44,000×	3 mm EE error (2.3%)	Time domain decoupled from network; closed-form gradients; MPC at 70 Hz on GPU
KNODE-Cosserat (Jiahao et al., 2024)	Hybrid physics + Neural ODE	Neural ODE residual	—	58.7% vs. physics-only	Coarse Cosserat backbone + learned correction; trained on PyElastica data
SoRoLEX (Berdica et al., 2024)	Learned simulator for RL	LSTM	∼10×	Task-space obs.	Full pipeline: sim data → surrogate → PPO; JAX GPU parallelization
PINN Soft Robot (Habich et al., 2025)	Physics-informed NN	PINN	467×	—	MPC at 47 Hz; as little as 1 training dataset; articulated soft robot
This work	Data-driven MLP surrogate	MLP (3×512)	∼46,000× (proj.)	*[Placeholder: —]*	Per-element CPG phase encoding; single-step MSE training; full 124-dim state

The most closely related prior work is SoRoLEX, which shares the same pipeline structure (collect trajectories, train neural dynamics model, run RL in learned environment). This work differs in architecture (MLP vs. LSTM), prediction target (full 124-dimensional Cosserat rod state vs. task-space observation), and the explicit use of a multi-step rollout loss.

2.2 RL Control for Snake and Soft Robots

Table 2 consolidates RL approaches for snake and soft robot control.

Table 2: RL methods for snake and soft robot control.

Reference	Robot	Actions	Algo.	Simulator	Key contribution
Shi et al. (2020)	3-link kinem.	Joint vel. (disc.)	DQN	Analytical	SE(2) fiber bundle reduces state 5D→3D
Liu et al. (2021)	4-link soft	CPG + option (6)	PPO	Custom	Multi-agent game for contact-aware nav.
Liu et al. (2022)	9-link wheeled	Gait offset (1)	PPO	MuJoCo	RL modulates undulation for path following
Liu et al. (2023)	4-link soft	CPG tonic (4)	PPO	MuJoCo	12-level curriculum + domain randomization
Zheng et al. (2022)	7-link rigid	CPG (5)	PPO	MuJoCo	Two-phase curriculum; series elastic act.
Jiang et al. (2024)	11-joint CO-BRA	Dual CPG (7)	DDPG	MuJoCo	A*→RL→CPG→PID hierarchy
Schaffer et al. (2024)	Biohybrid worm	Muscle act. (42)	PPO	PyElastica	Adaptive muscle force ceilings
Choi and Tong (2025)	Soft manip.	$\Delta\kappa$ (5)	SAC	DisMech	Implicit Euler for contact-rich tasks
Bing et al. (2019)	Soft snake	CPG (5)	PPO	PyElastica	CPG reduces exploration vs. per-joint
Hong et al. (2026)	Soft robot	Joint torques	PPO	FEM	FEM essential for sim-to-real fidelity
Janner et al. (2019)	Rigid artic.	Joint torques	SAC+M	MuJoCo	Model-based short-horizon planning
Sanchez-Gonzalez et al. (2020)	Particles	Forces	—	Custom	Graph network learns mesh-based physics
This work	Soft snake	CPG (5)	PPO	Elast./DM	Surrogate for 4–5 OOM speedup

Key themes: CPG-based action spaces dominate snake robot RL, reducing exploration burden relative to per-joint torque control. Curriculum learning and domain randomization bridge the sim-to-real gap. All approaches share the simulation cost bottleneck motivating this work: MuJoCo costs $\sim$1 ms/step, while Cosserat rod simulators cost 12–46 ms/step.

2.3 Physics Backend Comparison

This work uses two Cosserat rod simulators with fundamentally different numerical strategies. Table 3 summarizes the key differences.

Table 3: Comprehensive comparison of PyElastica and DisMech.

Property	**PyElastica**	**DisMech**
Solver		
Physics model	Cosserat rod theory	Discrete Elastic Rods (DER)
Integration scheme	Explicit symplectic (Verlet)	Implicit backward Euler (Newton)
Timestep per call	$\Delta t_{\text{sub}} = 0.001\,\text{s}$	$\Delta t = 0.05\,\text{s}$
Steps per RL action	500 substeps	1 step
Per-step cost	$\sim$46 ms (500 explicit evals)	$\sim$12 ms (1 Newton solve)
Supported forces		
Shear–stretch / Bending / Torsion	✓	✓
RFT (anisotropic drag)	✓	—
Self-contact	—	✓(FCL)
Control		
Curvature mechanism	Elastic rest shape (κ_0)	Hard constraint (natural strain)
CPG resolution	500 curvature updates/step	1 curvature snapshot/step
Metadata		
Theory	Cosserat rod	2008 DER (Bergou et al., 2008); 2010 (Bergou et al., 2010)
Simulator	2021 PyElastica (Naughton et al., 2021)	2024 DisMech (Choi et al., 2024)
Python framework	(same)	2025 Py-DiSMech (Lahoti et al., 2025)
Repository	`github.com/GazzolaLab/PyElastica`	`github.com/StructuresComp/dismech-rods`
Language	Python (Numba JIT)	C++ (Python bindings)

The fundamental trade-off: PyElastica's 500 substeps capture continuous CPG evolution but cost $4\times$ more wall-clock time. DisMech's single implicit step is faster and unconditionally stable, but collapses intra-step dynamics. For surrogate modeling, Elastica presents a harder approximation target.

The DisMech ecosystem has evolved significantly beyond the original DER formulation. Choi et al. (2024) introduced DisMech as a full simulator with implicit contact handling and real2sim validation, achieving an order-of-magnitude speedup over prior DER implementations. Lahoti et al. (2025) subsequently released Py-DiSMech, a vectorized Python reimplementation that further improves computational efficiency while adding PI-based shape control and modular actuation interfaces.

3 Neural Network Surrogate Model for Elastica

3.1 Motivation

Reinforcement learning for soft robots requires millions of environment interactions, yet first-principles Cosserat rod simulators are expensive: PyElastica costs $\sim$46 ms per step (500 Verlet substeps), while DisMech costs $\sim$12 ms (one Newton solve). At these rates, a 2×10^6-step training run takes $\sim$9 hours on Elastica and $\sim$36 hours with 16 parallel workers — prohibitive for hyperparameter sweeps or curriculum learning.

Neural surrogate models address this bottleneck by replacing the physics solver with a learned transition function f_θ that approximates the state-to-state mapping in a single GPU forward pass ($\sim$0.001 ms), targeting four-to-five orders-of-magnitude speedup. The approach is well-established for PDE systems: feedforward networks and neural ODEs have been used to learn dynamics for fluid simulation (Sanchez-Gonzalez et al., 2020), articulated soft robots (Habich et al., 2025), and Cosserat rods specifically (Berdica et al., 2024; Jiahao et al., 2024). The key challenge is *fidelity*: the surrogate must be accurate enough over multi-step rollouts that RL policies trained in the surrogate environment transfer to the true simulator.

This chapter presents our surrogate pipeline — physics, data collection, and model training: Cosserat rod PDE $\rightarrow$ staggered-grid ODE $\rightarrow$ explicit integrator $\rightarrow$ data $\rightarrow$ neural surrogate. Detailed derivations are in Appendix A. Data collection and surrogate training were performed on the hardware in Table 4.

Table 4: Hardware for data collection and surrogate training.

Component	**Specification**
GPU	1$\times$ Tesla V100-PCIE-16 GB (Volta, 5120 CUDA cores)
CPU	24 cores / 48 threads
RAM	270 GB
CUDA	12.8
Python	CPython 3.12

3.2 Elastica Physics

3.2.1 Cosserat Rod Formulation

The system is fully specified by the tuple $\mathcal{S} = (\mathcal{L},\ \Omega,\ \text{BC},\ \text{IC},\ \boldsymbol{\theta})$:

Table 5: Well-posedness tuple for the Elastica Cosserat rod system.

Component	Name	Specification
$\mathcal{L}$	Governing equations	Cosserat rod PDEs: conservation of linear and angular momentum with linear elastic constitutive law
Ω	Domain	$s \in [0, L]$, $L = 0.5\,\mathrm{m}$ (arc length) $\times\ t \in [0, T]$ (time)
BC	Boundary conditions	Free–free: $\mathbf{F} = \mathbf{M} = \mathbf{0}$ at $s = 0$ and $s = L$
IC	Initial conditions	Straight rod at rest: $\mathbf{x}(s, 0) = \mathbf{x}_0 + s\hat{\mathbf{e}}_x,\ \mathbf{v} = \boldsymbol{\kappa} = \boldsymbol{\omega} = \mathbf{0}$
$\boldsymbol{\theta}$	Parameters	E=10^5 Pa, ν=0.5, ρ=1200 kg/m^3, R=0.02 m, c_t=0.01, c_n=0.05, γ=0.002
Control	CPG forcing	$\mathbf{a} = (A, f, k, \phi_0, b) \in \mathbb{R}^5$ enters as time-varying rest curvature $\boldsymbol{\kappa}_0(s, t)$

The governing PDEs are:

$$\rho A \frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \qquad \text{(linear momentum)} \tag{1a}$$

$$\rho \mathbf{I} \frac{\partial \boldsymbol{\omega}}{\partial t} = \frac{\partial \mathbf{M}}{\partial s} + \mathbf{m}_{\text{ext}} + \frac{\partial \mathbf{x}}{\partial s} \times \mathbf{F} \qquad \text{(angular momentum)} \tag{1b}$$

with constitutive closure:

$$\mathbf{F} = \mathbf{S}\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_0), \quad \mathbf{M} = \mathbf{B}\,(\boldsymbol{\kappa} - \boldsymbol{\kappa}_0), \quad \mathbf{S} = \mathrm{diag}(GA, GA, EA), \quad \mathbf{B} = \mathrm{diag}(EI, EI, GJ) \tag{2}$$

Spatial discretization on a staggered grid (N=21 nodes, N_e=20 elements) reduces the PDE to 124 coupled first-order ODEs:

$$\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s},\, \mathbf{a},\, t), \qquad \mathbf{s} \in \mathbb{R}^{124}, \ \ \mathbf{a} \in \mathbb{R}^5 \tag{3}$$

Position Verlet ($\Delta t = 0.001\,\mathrm{s}$, $n_{\text{sub}} = 500$ substeps per RL step of $\Delta t_{\text{ctrl}} = 0.5\,\mathrm{s}$) produces the transition operator:

$$\mathbf{s}_{t+1} = T(\mathbf{s}_t,\, \mathbf{a}_t) = \underbrace{V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},499}) \circ \cdots \circ V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},0})}_{\text{500 substeps, each with distinct rest curvature}}(\mathbf{s}_t) \tag{4}$$

See Appendix A for detailed PDE derivations, staggered grid operators, external force handling, and the DisMech formulation.

Table 6: State vector $\mathbf{s}_t \in \mathbb{R}^{124}$ layout (Elastica).

Slice	Variable	Count	Meaning	Equation
`[0:21]`	x_i	21	Node x-positions (m)	Position kinematics
`[21:42]`	y_i	21	Node y-positions (m)	Position kinematics
`[42:63]`	$\dot{x}_i$	21	Node x-velocities ($\mathrm{m\,s^{-1}}$)	Linear momentum ODE
`[63:84]`	$\dot{y}_i$	21	Node y-velocities ($\mathrm{m\,s^{-1}}$)	Linear momentum ODE
`[84:104]`	ψ_e	20	Element yaw angle (rad)	Orientation kinematics
`[104:124]`	$\omega_{z,e}$	20	Element angular velocity ($\mathrm{rad\,s^{-1}}$)	Angular momentum ODE

3.2.2 Elastica Transition Operator

Algorithm 1 expands the full transition operator.

Algorithm 1 Elastica Transition Operator $T(\mathbf{s}_t,\, \mathbf{a}_t,\, t_{\text{step}}) \to \mathbf{s}_{t+1}$

Require: State $\mathbf{s}_t \in \mathbb{R}^{124}$, action $\mathbf{a}_t \in \mathbb{R}^5$, step time t_{step}
Ensure: Next state $\mathbf{s}_{t+1} \in \mathbb{R}^{124}$
1: $\mathbf{s} \leftarrow \mathbf{s}_t$
2: **for** $n = 0, 1, \ldots, 499$ **do**
3: $t^n \leftarrow t_{\text{step}} + n \cdot \Delta t$
4: Update rest curvature: $\kappa_v^{\text{rest}} \leftarrow A \sin(2\pi k \cdot s_v + 2\pi f \cdot t^n + \phi_0) + b$ for $v = 1, \ldots, 19$
5: Verlet half-step: $\mathbf{q}^{1/2} \leftarrow \mathbf{q} + \frac{1}{2}\Delta t\, \dot{\mathbf{q}}$
6: Compute geometry: edge vectors, tangents, strain, curvature
7: Compute internal forces (shear–stretch) and torques (bending)
8: Compute external forces: RFT friction + damping + gravity
9: Compute accelerations: $\ddot{\mathbf{x}} \leftarrow (\mathbf{F}^{\text{int}} + \mathbf{F}^{\text{ext}})/\mathbf{m}$
10: Verlet velocity update + full-step position update
11: **end for**
12: **return** $\mathbf{s}_{t+1} = (\mathbf{q},\, \dot{\mathbf{q}})$

Cost: $\sim$377,000 FLOPs/step ($\sim$46 ms on CPU). The CPG phase advances by $\Delta\phi = 2\pi f \Delta t$ per substep — the surrogate must learn the *integrated effect* of the traveling wave across 500 distinct curvature configurations.

3.2.3 CPG Control

The Central Pattern Generator drives locomotion by imposing time-varying rest curvature on the rod. When the rest curvature differs from the rod's current curvature, elastic bending moments drive the rod toward the target shape. As the CPG sweeps a sinusoidal curvature wave along the body, anisotropic ground friction (higher normal than tangential) converts the lateral undulation into net forward thrust—the same mechanism underlying biological serpentine locomotion. The RL agent selects five CPG wave parameters per control step:

Table 7: Action vector $\mathbf{a}_t \in \mathbb{R}^5$: CPG wave parameters.

Symbol	Name	Range	Units
A	Amplitude	$[0, 5]$	$\mathrm{rad\,m^{-1}}$
f	Frequency	$[0.5, 3.0]$	Hz
k	Wave number	$[0.5, 3.5]$	wavelengths/body
ϕ_0	Phase offset	$[0, 2\pi]$	rad
b	Turn bias	$[-2, 2]$	$\mathrm{rad\,m^{-1}}$

The serpenoid rest curvature at each Voronoi node:

$$\kappa_v^{\text{rest}}(t) = A \sin(2\pi k \cdot s_v + 2\pi f \cdot t + \phi_0) + b, \qquad v = 1, \ldots, 19. \tag{5}$$

3.3 Data Collection

Training data is generated by 16 independent PyElastica worker processes:

1. Actions sampled from 5D CPG space using Sobol quasi-random sequences (50% Sobol, 50% uniform).
2. 30% of episodes perturbed: random position offsets, angular velocity noise ($\sigma_\omega = 1.5\,\mathrm{rad/s}$).
3. Each worker stores transitions $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}, t_{\text{start}})$ to batch files.
4. Health monitor respawns stalled workers.

Table 8: Data collection pipeline parameters.

Parameter	**Value**
Parallel workers	16
Transition size (with forces)	$\sim$2055 bytes
Target dataset size	$\sim$10 GB ($\sim$5M transitions)
Per-worker throughput	$\sim$22 transitions/s
Aggregate throughput	$\sim$350 transitions/s

Dataset quality. The final dataset contains 4,336,600 single-step transitions. Table 9 summarizes automated validation.

Table 9: Data quality validation results.

Metric	**Value**	**Threshold**	**Status**
NaN/Inf rate	0.00%	$< 0.1\%$	Pass
Duplicate transitions	3.84%	$< 1\%$	Fail
Constant features	0	0	Pass
Outlier rate ($> 5\sigma$)	0.096%	$< 0.5\%$	Pass
Action 5D fill fraction	74.5%	$> 5\%$	Pass
Per-dim action fill	100%	$> 90\%$	Pass

The 3.84% duplicate rate arises from identical initial conditions under 1-step episodes and amounts to implicit oversampling — retained without deduplication (4,170,082 unique transitions).

3.4 Model Architecture and Training

3.4.1 Input Representation

The surrogate input concatenates three components:

$$\mathbf{z}_t = \left[\bar{\mathbf{s}}_t \parallel \mathbf{a}_t \parallel \boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{N_e}\right] \in \mathbb{R}^{189} \tag{6}$$

Table 10: Surrogate input decomposition ($189 = 124 + 5 + 60$).

Component	Dim	Description
$\bar{\mathbf{s}}_t$	124	Z-score normalized rod state
$\mathbf{a}_t$	5	Normalized CPG action $\in [-1, 1]^5$
$\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{20}$	60	Per-element phase triplets

3.4.2 Per-Element CPG Phase Encoding

For element e, the phase triplet is:

$$\boldsymbol{\phi}_e = \left(\sin(2\pi f \cdot t_e),\ \cos(2\pi f \cdot t_e),\ \kappa_e^{\text{actual}}\right), \qquad t_e = \frac{k \cdot s_e}{f} \tag{7}$$

This provides the exact phase of the serpenoid wave at each element's location, plus the instantaneous curvature κ_e^{actual} for the control error. Phase features are computed on-the-fly from stored action and t_{start}.

An earlier V1 global phase encoding ($\mathbb{R}^2$) achieved only $R^2 = 0.23$ for angular velocity; the V2 per-element encoding significantly improved prediction.

3.4.3 Architecture and Prediction

The surrogate predicts a normalized state delta:

$$f_\theta(\mathbf{z}_t) \approx T(\mathbf{s}_t, \mathbf{a}_t) - \mathbf{s}_t, \qquad \hat{\mathbf{s}}_{t+1} = \mathbf{s}_t + \boldsymbol{\sigma}_\Delta \odot f_\theta(\mathbf{z}_t) + \boldsymbol{\mu}_\Delta \tag{8}$$

Network architecture:

$$\mathbb{R}^{189} \xrightarrow{\text{Linear}} \big[\text{Linear}(512) \to \text{LayerNorm} \to \text{SiLU}\big]^{\times 3} \xrightarrow{\text{Linear}} \mathbb{R}^{124} \tag{9}$$

- Output layer initialized to zero (default prediction: mean empirical transition)
- Total trainable parameters: $\sim$659K
- Delta prediction concentrates the target distribution and provides an implicit residual connection

[Placeholder: Surrogate architecture diagram showing input decomposition, MLP layers, and affine reconstruction.]

3.4.4 Training

The surrogate is trained with a single-step MSE loss on normalized state deltas:

$$\mathcal{L}_{\text{single}} = \frac{1}{N} \sum_{i=1}^{N} \big\| f_\theta(\mathbf{z}_t^{(i)}) - \overline{\Delta \mathbf{s}}_{\text{true}}^{(i)} \big\|^2 \tag{10}$$

Density-weighted sampling. Each training sample is weighted by inverse estimated local density:

$$w^{(i)} = \frac{1}{\hat{p}(\mathbf{c}^{(i)})}, \qquad \mathbf{c}^{(i)} = \big[\bar{x}_{\text{CoM}},\ \bar{y}_{\text{CoM}},\ \|\dot{\mathbf{x}}\|,\ |\bar{\omega}_z|\big] \tag{11}$$

Weights are clipped at $10\times$ the mean to prevent rare samples from dominating.

3.4.5 Hyperparameters

Table 11: Surrogate training hyperparameters.

Hyperparameter	**Value**
Architecture	4-layer MLP (4×1024)
Activation / Normalization	SiLU / LayerNorm (pre-activation)
Optimizer	Adam (β_1=0.9, β_2=0.999)
Learning rate	1×10^{-4} (cosine, 5-epoch warmup)
Batch size	4096 (auto-scaled to $\sim$245K effective)
Mixed precision	bfloat16
Early stopping	Patience 30 (validation MSE)
Total parameters	$\sim$3.43M

Table 12: Per-run configuration differences and results.

	Run A	**Run B (best)**	**Run C**
State dim	128	128	130
Dropout	0.0	0.1	0.1
Weight decay	10^{-5}	10^{-4}	10^{-4}
Epochs trained	80	110	57
Best val. loss	0.712	**0.601**	0.733
Overall R^2	—	**0.393**	0.258

3.5 Results

The best surrogate (Run B) reached a validation loss of 0.601 and an overall R^2 of 0.393 after 110 epochs on 3.9M transitions.

Table 13: Surrogate single-step prediction accuracy by state component (Run B, epoch 110).

State component	R^2	MSE
CoM position ($\bar{x}, \bar{y}$)	0.852	2.81×10^{-7}
Relative positions ($\Delta x_i, \Delta y_i$)	0.773	$\sim 3.2 \times 10^{-5}$
Angular velocity ($\omega_{z,e}$)	0.320	1.442
Heading (θ)	0.287	2.49×10^{-3}
Yaw (ψ_e)	0.173	4.78×10^{-2}
Velocities ($\dot{x}_i, \dot{y}_i$)	0.145	8.10×10^{-2} / 2.13×10^{-1}
Overall	**0.393**	0.601

Geometric quantities (CoM, relative positions) converge within 20 epochs. Velocity and angular components improve gradually and have not plateaued by epoch 110, suggesting longer training or architectural changes could help.

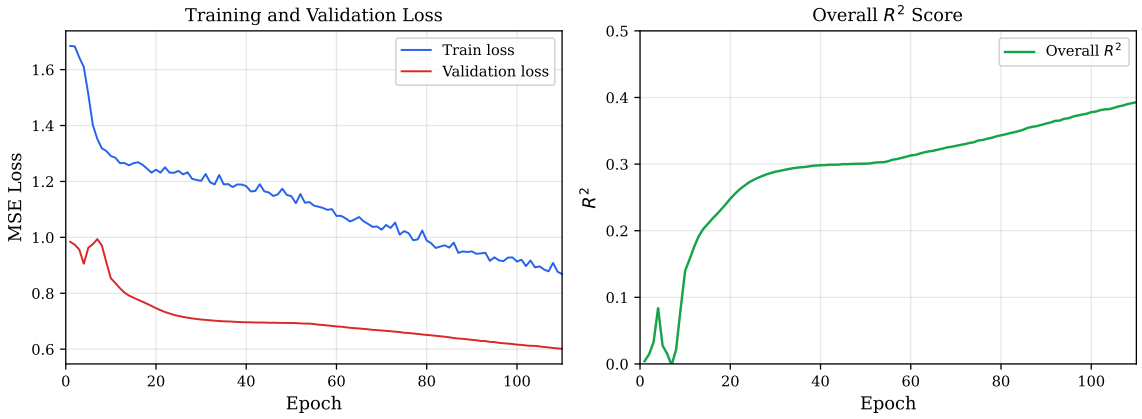


Figure 1: Training and validation MSE loss (left) and overall R^2 (right) over 110 epochs for Run B. The persistent train–validation gap indicates mild overfitting despite dropout and weight decay.

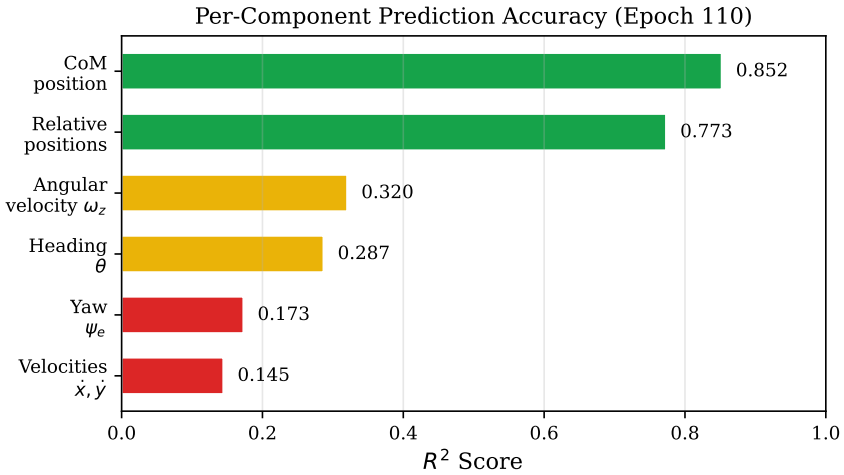


Figure 2: Per-component R^2 at epoch 110 (Run B). Geometric quantities are well predicted; velocity components remain challenging.

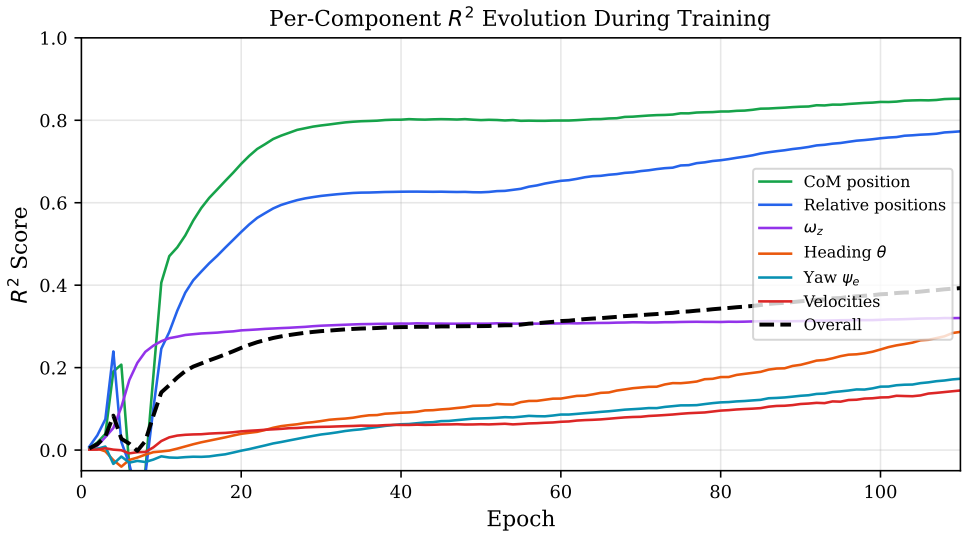


Figure 3: Per-component R^2 evolution during training. Position components converge within 20 epochs; angular and velocity components improve gradually.

3.6 Challenges

Angular velocity prediction. Angular velocity $\omega_{z,e}$ is the weakest predicted component ($R^2 = 0.320$). The control step $\Delta t = 0.5$ s spans approximately 1.5 CPG cycles at typical frequencies, during which ω_z reverses sign multiple times. A single-step MLP must therefore predict the *net* angular velocity after integrating through these oscillations—a many-to-one mapping that averages away the within-step dynamics. Per-element CPG phase encoding (Section **??**) partially mitigates this by informing the model where in the oscillation cycle each element begins, but the fundamental aliasing remains.

Training data distribution bias. Data collected from policy rollouts clusters around learned behaviors, leaving large regions of state-action space unrepresented. The rod always starts from a straight (zero curvature) configuration, so the surrogate rarely observes transitions from highly bent states encountered during locomotion. Sobol quasi-random action sampling, initial state perturbation, and curvature noise injection (Section 3.3) address this, but full coverage of the 189-dimensional input space remains intractable.

Curvature update granularity. The serpenoid rest curvature was initially applied once per control step (every 0.5 s), while the physics integrator advances in 1 ms substeps. This created a discontinuous forcing signal: the rod experienced a step change in target curvature at the start of each control interval, then evolved under constant curvature for 500 substeps. Moving the curvature update into the substep loop produces a near-continuous traveling wave, improving both the physical realism of the training data and surrogate prediction accuracy.

Persistent overfitting. Despite dropout ($p = 0.1$) and weight decay ($\lambda = 10^{-4}$), a train–validation gap persists throughout training (Figure **??**). The gap suggests that the 4.3M-transition dataset, while large, does not fully cover the state-action manifold—the model memorizes training trajectories rather than learning generalizable dynamics. Increasing dataset diversity (more perturbation variance, longer episodes) or architectural changes (e.g., attention over spatial elements) may help close this gap.

4 Reinforcement Learning on Elastica

4.1 Environment and Training Setup

Direct RL training uses the Elastica physics simulator as the environment, running on 1× Tesla V100-PCIE-16 GB (24 cores, 270 GB RAM, CUDA 12.8). At each step, the agent selects CPG parameters $\mathbf{a}_t \in [-1, 1]^5$, the simulator advances via 500 substeps, and a displacement-based reward is computed:

$$r_t = x_{\text{CoM}}(t+1) - x_{\text{CoM}}(t) \tag{12}$$

PPO is used as the policy optimization algorithm with TorchRL.

Table 14: PPO training hyperparameters (Session 10, best).

Hyperparameter	**Value**
Parallel environments	16
Total frames	2×10^6
Frames per batch	8192
Mini-batch size	256
PPO clip ϵ / PPO epochs	0.2 / 4
Learning rate	1×10^{-4} (linear decay to 10^{-5})
Discount γ / GAE λ	0.99 / 0.95
Entropy coefficient / Value coefficient	0.02 / 0.5
Max gradient norm / Target KL	0.5 / 0.01
Max episode steps	500

4.2 Direct Simulation Results

Training exhibits three phases: (1) *learning* (0–164k steps, peak rolling average 140.0), (2) *decline* (164k–574k, policy overshooting in CPG space), (3) *steady state* (574k–2M, stabilized ∼102). Best checkpoint reward: 156.66 at step 155,648.

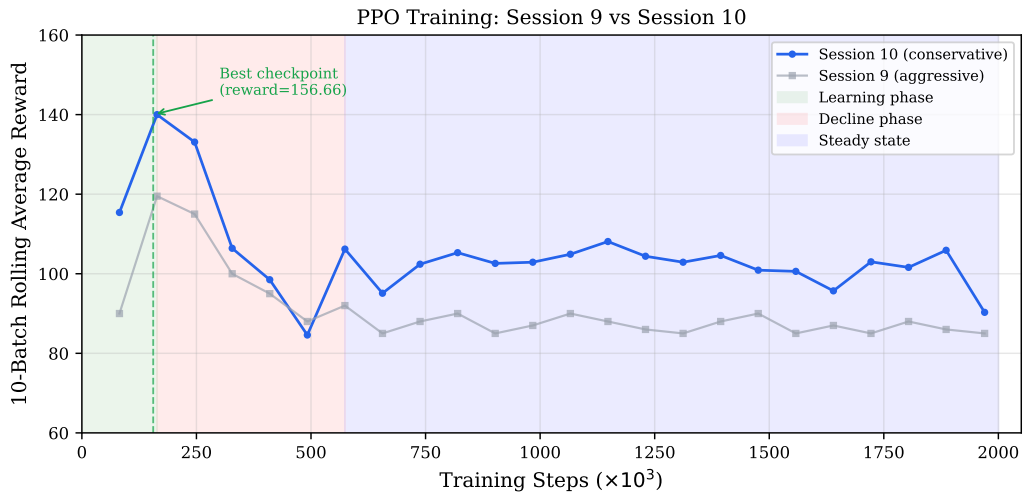


Figure 4: PPO training curves. Session 10 (conservative hyperparameters) achieves higher peak reward and more stable behavior than Session 9 (aggressive hyperparameters).

Evaluation. 20 deterministic-action episodes with randomized initial headings:

Table 15: Direct Elastica RL training and evaluation summary.

Metric	**Value**
Training time	$\sim$9 h
Throughput	$\sim$55–60 FPS
Peak rolling reward	140.0
Best checkpoint reward	156.66
Eval mean displacement	1.495 ± 0.13 m
Eval mean reward	64.90 ± 1.44
Goal success rate	0/20 (0%)

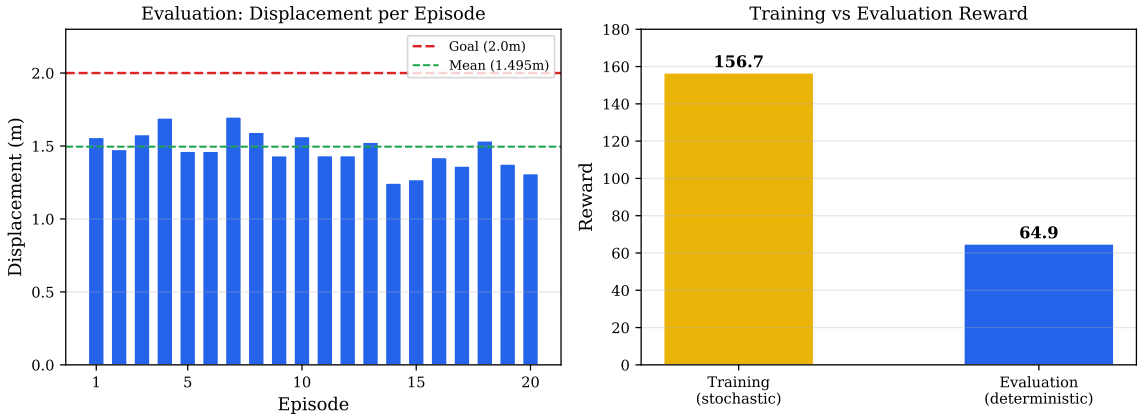


Figure 5: Evaluation of best Session 10 checkpoint. Left: displacement per episode. Right: training vs. evaluation reward gap.

The 2.4× gap between training (156.66) and evaluation (64.90) rewards is attributed to stochastic actions during training, favorable heading distributions, and goal-reaching bonuses.

4.3 Surrogate Environment Results

Surrogate-based RL training has not yet been conducted. The moderate overall $R^2 = 0.393$ and poor velocity prediction ($R^2 = 0.145$) raise concerns about multi-step rollout fidelity. The key risk is that the policy may exploit surrogate artifacts rather than learning physically grounded gaits that transfer to the Elastica simulator.

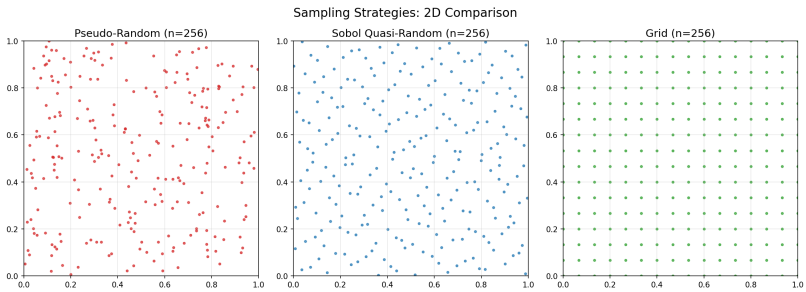


Figure 6: Action-space sampling comparison: Sobol quasi-random vs. pseudo-random coverage for surrogate data collection.

4.4 Challenges

- **Computational cost:** $\sim$46 ms/step × 16 workers = $\sim$350 steps/s; 2×10^6 steps requires $\sim$36 h. DisMech is $\sim$4× faster but still CPU-bound.
- **Exploration in CPG space:** Large regions produce no locomotion (zero amplitude, extreme frequencies). Curriculum learning over parameter subsets improves efficiency.
- **Reward sparsity:** Distance-based rewards require discovering forward-moving gaits before receiving signal.
- **Backend discrepancy:** DisMech-trained policies do not transfer to Elastica due to friction, curvature mechanism, and temporal resolution differences.

5 Reinforcement Learning on DisMech (Choi2025 Follow Task)

This chapter presents PPO experiments on the DisMech physics backend for the `follow_target` task from Choi and Tong (2025). While the original paper uses a Gymnasium-based environment with a custom training framework, our implementation uses TorchRL for both the environment wrapper (`EnvBase`) and the training loop (`SyncDataCollector`, `ParallelEnv`). We conduct a controlled ablation study over four reward-shaping configurations, using per-component reward logging and a common fair metric to enable valid cross-run comparisons. All experiments ran on 2× RTX A4000 16 GB (Ampere, 6144 CUDA cores each), 16 cores / 32 threads, 270 GB RAM, CUDA 12.8.

5.1 Task Description

The `follow_target` task requires a clamped soft manipulator (Cosserat rod, 21 nodes, 1 m length) to track a moving target in 3D. The target moves at 0.05 m/s within a workspace of radius 0.3–0.9 m. At each control step, the agent selects 5 curvature increments $\Delta\kappa \in [-1, 1]$, applied as natural strain through DisMech's bend springs.

Table 16: Environment configuration (common to all runs).

Parameter	**Value**
Physics backend	DisMech (C++ implicit Euler)
Rod discretization	21 nodes, 20 elements
Observation dim	34 (positions, curvatures, target)
Action dim	5 ($\Delta\kappa$ curvature increments)
Max episode steps	200
Control timestep	0.05 s (2 substeps at dt=0.05 s)
Target speed	0.05 m/s
Workspace radius	0.3–0.9 m

5.2 Reward Design

Choi and Tong (2025) define the follow-target reward with two terms:

$$r(s, s') = \underbrace{\exp\big(-5\, d(s')\big)}_{r_{\text{dist}}} + \underbrace{10\,\big(d(s) - d(s')\big)}_{r_{\text{improve}}}, \qquad d(s) = \|\mathbf{x}_{\text{tip}} - \mathbf{x}_{\text{target}}\|. \tag{13}$$

The first term is a bounded distance reward in $[0, 1]$, approaching 1 as the tip reaches the target. The second is an improvement bonus proportional to the step-wise distance reduction. The scaling factor of 5 was chosen to match the workspace radius; the improvement multiplier of 10 amplifies the approach signal.

Simplification for PPO. The original reward was designed for SAC, which averages over a replay buffer and tolerates high reward variance. For on-policy PPO, the improvement bonus with its $10\times$ multiplier creates a reward SNR of only 0.08 (the signal is 8% of the noise), causing the value function to fail. We therefore use only the distance component as the base reward:

$$r_{\text{dist}}(s) = \exp\big(-5\, d(s)\big). \tag{14}$$

This raised the SNR from 0.08 to 1.02 (a $13\times$ improvement) and stabilized explained variance from 0.24 to 0.65. We then investigate three augmentations to this simplified base reward, described below.

Heading reward. A cosine-similarity bonus for pointing the tip toward the target:

$$r_{\text{heading}}(s) = \frac{1 + \cos\theta}{2}, \qquad r_{\text{total}} = (1 - w)\, r_{\text{dist}} + w\, r_{\text{heading}}, \tag{15}$$

where θ is the angle between the tip tangent and the tip-to-target direction, and $w \in [0, 1]$ is a mixing weight. Both components lie in $[0, 1]$, so the total reward is bounded.

Potential-Based Reward Shaping (PBRS). Following Ng et al. (1999), we define a potential $\Phi(s) = -d(s)$ and add the shaping reward:

$$F(s, s') = \gamma_{\text{pbrs}}\, \Phi(s') - \Phi(s) = d(s) - \gamma_{\text{pbrs}}\, d(s'), \tag{16}$$

where γ_{pbrs} is a discount factor (we use 0.99). This is guaranteed not to change the optimal policy. When combined with the base reward: $r_{\text{total}} = r_{\text{dist}} + F(s, s')$.

PBRS-only. An ablation where the base reward is removed entirely, using only the shaping signal: $r_{\text{total}} = F(s, s')$.

Curriculum learning. All runs use curriculum learning: target speed ramps from 20% to 100% over the first 100 episodes per worker, avoiding reward-scale shock at initialization.

Fair comparison metric. Different reward functions produce incomparable total rewards (e.g., heading inflates the total by up to 72%). We log per-component rewards and use $r_{\text{dist}} = \exp(-5d)$ as the *fair metric* for cross-run comparison, since it depends only on the physical outcome (distance to target) and is independent of the reward function.

5.3 Experimental Setup

All four runs use identical PPO hyperparameters, differing only in the reward function. Table 17 shows the configuration of each run.

Table 17: Run configurations for the reward-shaping ablation study.

	Baseline	**Heading**	**PBRS+Base**	**PBRS-Only**
Algorithm	PPO	PPO	PPO	PPO
Network	4×512 ReLU	4×512 ReLU	4×512 ReLU	4×512 ReLU
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Clip ϵ	0.2	0.2	0.2	0.2
Epochs per batch	10	10	10	10
Mini-batch size	1024	1024	1024	1024
Frames per batch	8192	8192	8192	8192
Entropy coeff	0.1	0.1	0.1	0.1
Parallel envs	100	100	100	100
Total frames	5M	5M	5M	5M
Seed	42	42	42	42
Curriculum	Yes	Yes	Yes	Yes
Reward-specific parameters				
Base reward r_{dist}	Yes	Yes	Yes	No
Heading weight w	0.0	0.3	0.0	0.0
PBRS γ	0.0	0.0	0.99	0.99

5.4 Results

Table 18 summarizes the training outcomes. All metrics in the "fair metric" column use $r_{\text{dist}} = \exp(-5d)$ averaged over the final 20% of training, enabling direct comparison regardless of the reward function. The signal-to-noise ratio (SNR) is computed as $|\bar{r}|/\sigma_r$ over all training batches.

Table 18: Training results on follow_target (5M frames each). Fair metric is $r_{\text{dist}} = \exp(-5d)$ averaged over the final 20% of episode entries.

Run	r_{dist} **(fair)**	**vs. Baseline**	**SNR**	**Episode Reward**	**Best Episode**
Baseline	0.094	—	1.20	18.7	61.3
Heading (w=0.3)	0.072	−23.3%	1.82	39.1	63.0
PBRS+Base	0.109	+16.9%	1.00	27.4	77.9
PBRS-Only	0.067	−28.7%	0.10	1.8	2.9

Key observations:

- **PBRS+Base is the best performer** at +16.9% over baseline on the fair metric, with the

highest best-episode reward (77.9). The shaping signal $F = d(s) - 0.99\, d(s')$ provides a per-step gradient toward the target, accelerating learning without distorting the optimal policy.

- **Heading reward hurts by 23.3%.** Although the heading run achieves inflated episode rewards (39.1 vs. 18.7), this reflects the heading component contributing ∼72% of the total reward. On the fair metric, heading reduces distance-tracking performance—a case of reward hacking where the agent optimizes alignment at the expense of proximity.
- **PBRS-only is insufficient** (−28.7%). Removing the base reward leaves only the shaping signal, which has critically low SNR (0.10, vs. the 0.5 threshold for reliable learning). The value function achieves high explained variance (0.88) because near-zero returns are easy to predict, but this does not help the policy distinguish good states from bad.
- **SNR predicts performance.** The ranking by fair metric (PBRS+Base > Baseline > Heading > PBRS-Only) aligns with SNR when accounting for reward hacking: heading has higher SNR (1.82) but the signal points in the wrong direction.

Reward curves. Figure 7 shows the fair metric and distance to target over training. PBRS+Base (green) separates from the other runs after ∼2M frames, with the improvement still accelerating at 5M frames. The baseline (blue) and heading (orange) both improve slowly; PBRS-Only (red, dashed) remains flat near the bottom. Note: the baseline curve has no distance-to-target data because per-component logging was added after that run.

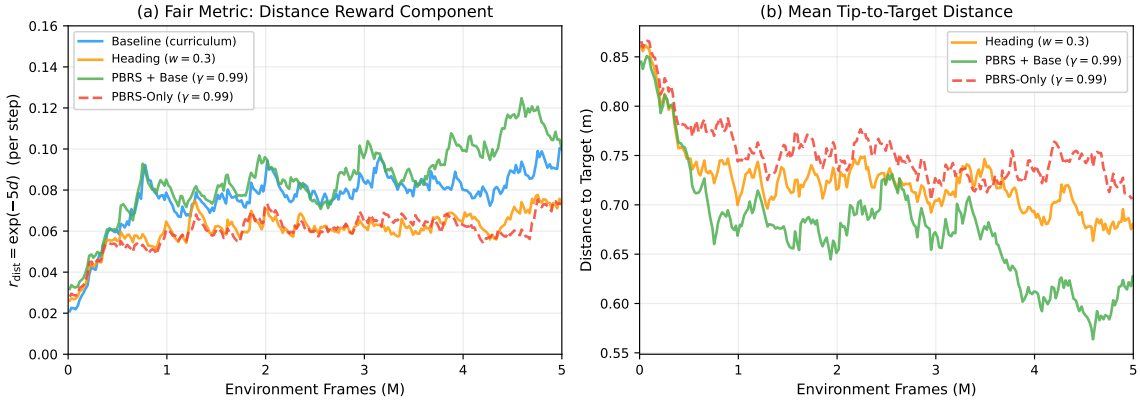


Figure 7: PPO reward-shaping ablation on follow_target (5M frames, 100 envs, seed 42). (a) Fair metric $r_{\text{dist}} = \exp(-5d)$ per step, EMA-smoothed ($\alpha = 2/41$). (b) Mean tip-to-target distance. PBRS+Base (green) outperforms all variants; heading (orange) and PBRS-Only (red) underperform baseline.

5.5 SAC Gradient Instability

In separate experiments, we trained SAC (20M frames, 100 envs) and PPO (50M frames, 100 envs) on the same task using the paper-matching 3×256 ReLU network. SAC reached a

best reward of 73.84 early ($\sim$2M frames) but was unable to sustain performance, while PPO plateaued at $\sim$17 average reward. Figure 8 reveals the root cause: without entropy regularization ($\alpha = 0$) and without gradient clipping, SAC's actor gradient norm grows from $\sim$0.04 to $\sim 1.5 \times 10^{10}$ over 20M frames, while the critic gradient norm remains bounded at $\sim$1–40. PPO gradient norms stay bounded at 2–28 throughout, consistent with its clipped surrogate objective.

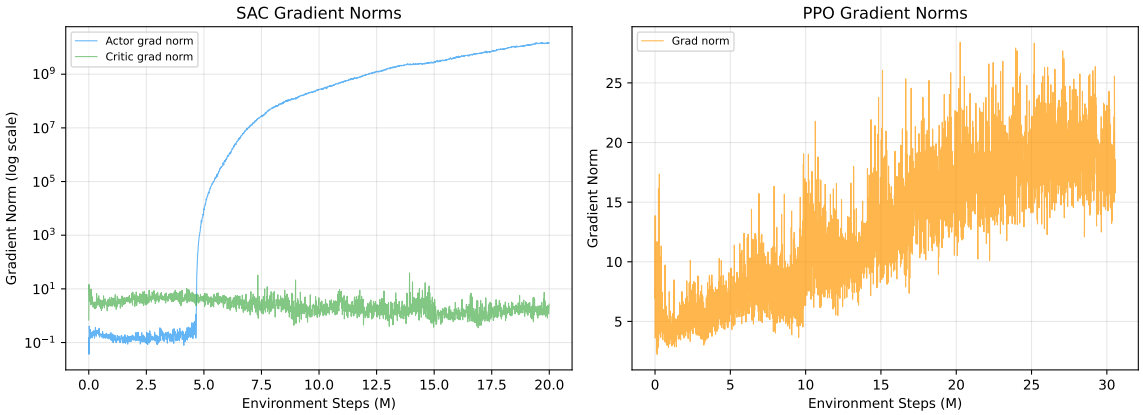


Figure 8: Gradient norms for SAC (left, log scale) and PPO (right, linear scale). SAC actor gradients explode by 10 orders of magnitude; PPO gradients remain stable.

The paper's 5M frame budget may mask this instability, as the collapse onset occurs at $\sim$3–5M frames. Possible mitigations include mild entropy regularization ($\alpha > 0$), actor-specific gradient clipping, or learning rate scheduling.

5.6 Discussion

The ablation study demonstrates that reward design has a larger effect on learning than hyperparameter tuning in this task. PBRS provides a principled mechanism for accelerating learning without changing the optimal policy, but requires the base reward to define *which* states are desirable—the shaping signal only provides directional information (closer vs. farther).

The heading reward, despite intuitive appeal, exemplifies a common RL pitfall: auxiliary objectives can dominate the total reward and redirect optimization away from the true task metric. Per-component logging is essential for diagnosing such issues, since the aggregate episode reward (39.1 for heading vs. 18.7 for baseline) misleadingly suggests heading is superior.

The PBRS+Base improvement (+16.9%) is still accelerating at 5M frames, suggesting longer training or higher frame budgets may yield further gains. A natural next step is combining PBRS with entropy scheduling or adaptive γ to further improve sample efficiency.

6 Physics-Informed Neural Network

All PINN experiments ran on the same hardware as Section 5: 2× RTX A4000 16 GB (Ampere), 16 cores, 270 GB RAM, CUDA 12.8.

6.1 Mathematical Formulation

6.1.1 Standard PINN

Physics-informed neural networks (Raissi et al., 2019) augment the data-driven loss with a physics residual derived from the governing equations. Given the state-space model $\dot{\mathbf{x}}(t) = \mathbf{f}_{\text{SSM}}(\mathbf{x}(t), \mathbf{u}(t))$, a network $\mathbf{h}_\theta$ predicts $\hat{\mathbf{x}}_t = \mathbf{h}_\theta(\mathbf{x}_0, \mathbf{u}, t)$. The PINN loss:

$$\mathcal{L}(\theta) = \underbrace{\lambda_{\text{IC}} \left\| \hat{\mathbf{x}}_0 - \mathbf{x}_0 \right\|^2}_{\mathcal{L}_{\text{IC}}} + \underbrace{\lambda_{\text{phys}} \frac{1}{N_c} \sum_{k=1}^{N_c} \left\| \frac{\partial \hat{\mathbf{x}}_{t_k}}{\partial t} - \mathbf{f}_{\text{SSM}}(\hat{\mathbf{x}}_{t_k}, \mathbf{u}_{t_k}) \right\|^2}_{\mathcal{L}_{\text{phys}}} + \underbrace{\lambda_{\text{data}} \frac{1}{N_d} \sum_{k=1}^{N_d} \left\| \hat{\mathbf{x}}_{t_k} - \mathbf{x}_{\text{data},k} \right\|^2}_{\mathcal{L}_{\text{data}}} \tag{17}$$

The time derivative $\partial \hat{\mathbf{x}}_t / \partial t$ is obtained via autodiff through the network, doubling the computational cost per collocation point.

Practical bottlenecks:

- **Expensive autodiff** — additional backward pass per collocation point
- **Loss balancing** — three competing objectives require careful weighting
- **Training instability** — can diverge for systems with $m \gg 1$ coupled states

6.1.2 Physics Regularizer

The simplest physics-informed approach adds algebraic constraints as soft regularizers. Four constraints are enforced via trapezoidal integration on the 124-dimensional state delta $\Delta\mathbf{x} = \mathbf{x}_{t+1} - \mathbf{x}_t$:

Table 19: Physics regularizer constraints.

ID	Constraint	Formulation
C1	Translational kinematics	$\Delta x_i \approx (v_{x,i} + \frac{1}{2}\Delta v_{x,i})\Delta t$
C2	Angular kinematics	$\Delta\psi_e \approx (\omega_{z,e} + \frac{1}{2}\Delta\omega_{z,e})\Delta t$
C3	Curvature–moment	Spatial gradient consistency (0.1× weight)
C4	Bounded kinetic energy	$\text{ReLU}(\lvert\Delta\text{KE}\rvert - E_{\text{thr}})^2$

Total regularization: $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{kin},x} + \mathcal{L}_{\text{kin},y} + \mathcal{L}_{\text{ang}} + \mathcal{L}_{\text{curv}} + \mathcal{L}_{\text{energy}}$.

6.1.3 DD-PINN Method

The domain-decoupled PINN (Krauss et al., 2024; Licher et al., 2025a) resolves all three PINN bottlenecks: **remove time as a network input** and output parameters of an analytically differentiable ansatz.

$$\boxed{\hat{\mathbf{x}}_t = \mathbf{g}(\mathbf{a}_{\text{ans}}, t) + \mathbf{x}_0, \qquad \mathbf{a}_{\text{ans}} = \mathbf{f}_{\text{NN}}(\mathbf{x}_0, \mathbf{u}_0;\, \theta)} \tag{18}$$

The ansatz for each state variable j:

$$g_j(t) = \sum_{i=1}^{n_g} \alpha_{ij}\, e^{-\delta_{ij} t} \big[\phi_g(\beta_{ij}\, t + \gamma_{ij}) - \phi_g(\gamma_{ij})\big] \tag{19}$$

where $\phi_g(x) = \sin(x)$ and the subtraction ensures $g_j(0) = 0$ exactly.

Key advantages over standard PINN:

- **Exact IC:** $\mathbf{g}(\mathbf{a}_{\text{ans}}, 0) \equiv \mathbf{0}$ by construction
- **Closed-form gradients:** $\dot{\mathbf{g}}$ computed analytically — no autodiff cost
- **Simplified loss:** only 2 terms ($\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$)
- **More collocation points:** 10–25$\times$ more than standard PINNs due to cheap evaluation

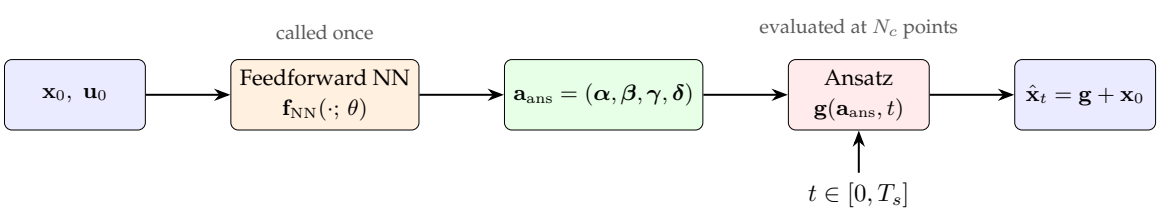


Figure 9: DD-PINN inference pipeline. The network is evaluated once to produce ansatz parameters; the ansatz is then evaluated at arbitrarily many time points.

6.1.4 Limitations of PINNs

PINNs face fundamental limitations for complex dynamical systems:

- **Spectral bias:** Networks learn low-frequency components first, making high-frequency CPG dynamics difficult.
- **Scaling to high dimensions:** Standard PINNs struggle for $m \gg 1$ (the snake robot has $m = 124$).
- **Discontinuous physics:** Contact and friction forces create non-smooth dynamics that PINNs cannot easily enforce.
- **Ansatz design:** DD-PINNs require an appropriate ansatz function designed a priori for the dynamics class.

6.2 PDE System for Snake Robot

6.2.1 Differentiable Cosserat Rod Right-Hand Side

A differentiable implementation of $\mathbf{f}_{\text{SSM}}$ computes $\dot{\mathbf{x}} = \mathbf{f}_{\text{SSM}}(\mathbf{x})$ for the 124-dimensional state vector using an inextensible rod approximation (bending + RFT friction, no stretching). The right-hand side assembles six blocks:

$$\mathbf{f}_{\text{SSM}}(\mathbf{x}) = \big(\dot{x}_i,\ \dot{y}_i,\ \dot{v}_{x,i},\ \dot{v}_{y,i},\ \dot{\psi}_e,\ \dot{\omega}_{z,e}\big) \in \mathbb{R}^{124} \tag{20}$$

Key components:

- **Kinematics:** $\dot{x}_i = v_{x,i}$, $\dot{\psi}_e = \omega_{z,e}$
- **Bending moments:** $M_j = EI(\psi_{j+1} - \psi_j)/\Delta\ell$
- **RFT friction:** $\mathbf{F}_i^{\text{fric}} = -c_t \mathbf{v}_{\text{tan},i} - c_n \mathbf{v}_{\text{norm},i}$ with ε-regularized tangents
- **Newton's 2nd law:** $\dot{v}_{x,i} = (F_{x,i}^{\text{fric}} + F_{x,i}^{\text{elastic}})/m_i$

Table 20: Physics fidelity: Elastica simulator vs. DD-PINN RHS vs. regularizer.

Physics term	Elastica	DD-PINN RHS	Regularizer
Kinematics	Exact (Verlet)	Exact	Soft penalty (C1, C2)
Shear–stretch	Full	Omitted (inextensible)	Omitted
Bending moments	Full 3D, director-based	2D simplified FD	Approximate (C3)
RFT friction	Anisotropic + damping	Anisotropic, no damping	Not modelled
Differentiable?	In principle (gradient explosion at 500 steps)	Yes	Yes
Cost	∼46 ms (CPU)	∼0.3 ms (GPU)	∼0.1 ms (GPU)

6.2.2 Physics-Regularized Loss

For the regularized surrogate:

$$\mathcal{L}^{(n)}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda\, \sigma_{\text{reg}}(n)\, \mathcal{L}_{\text{reg}}(\theta) \tag{21}$$

with piecewise-linear warmup (20% data-only).

6.2.3 DD-PINN Loss and Training

The DD-PINN training loss at epoch n:

$$\boxed{\mathcal{L}^{(n)}(\theta) = w_{\text{data}}^{(n)}\, \mathcal{L}_{\text{data}}(\theta) + w_{\text{phys}}^{(n)}\, \lambda\, \sigma(n)\, \mathcal{L}_{\text{phys}}(\theta)} \tag{22}$$

where $w_{\text{data}}^{(n)}, w_{\text{phys}}^{(n)}$ are ReLoBRaLo (Bischof and Kraus, 2022) adaptive weights, λ is fixed physics strength, and $\sigma(n)$ is a curriculum schedule (15% warmup, 30% ramp, full physics

at 45%).

6.2.4 Nondimensionalization

Physics-based reference scales keep the residual at $\mathcal{O}(1)$:

Table 21: Nondimensionalization scales.

Scale	Value	Source
L_{ref}	1.0 m	Rod length
t_{ref}	0.5 s	Control timestep
V_{ref}	2.0 m/s	L/t
ω_{ref}	2.0 rad/s	$1/t$
F_{ref}	EI/L^2	Elastic force

6.2.5 DD-PINN Configuration

Table 22: DD-PINN training configuration.

Parameter	Value
State dimension	124
Ansatz basis functions	sweep $\{5, 7, 10\}$
Hidden layers	4×512
Fourier features	128, $\sigma = 10.0$
Collocation points	100 000 (Sobol)
Physics weight λ	1.0
Batch size / Learning rate	4 096 / 10^{-3} (cosine)
Curriculum warmup	15% data-only

6.3 Results and Comparison

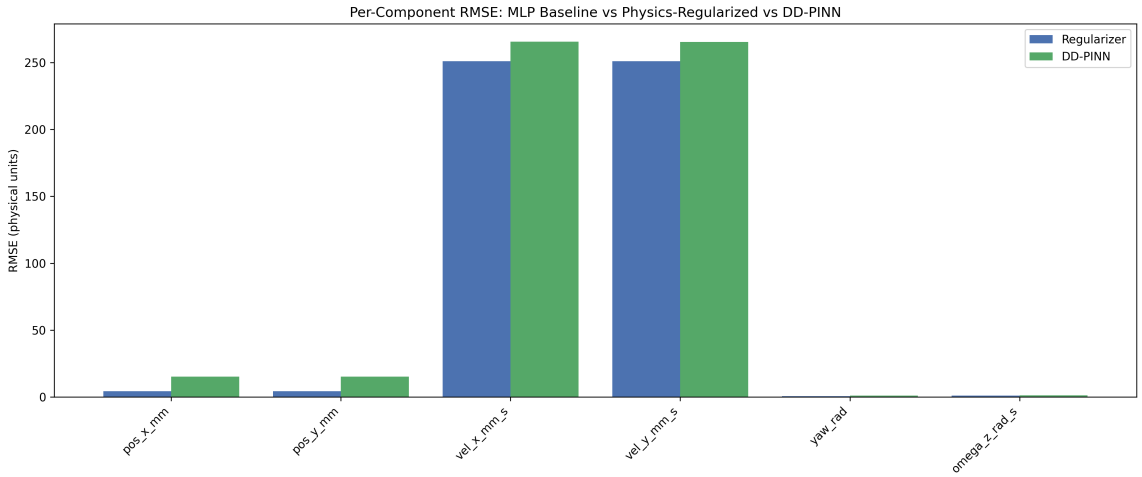


Figure 10: Per-component RMSE comparison across surrogate approaches. Physics-regularized training ($\lambda = 0.01$) converged in 138 epochs ($\sim$8 h on RTX A4000).

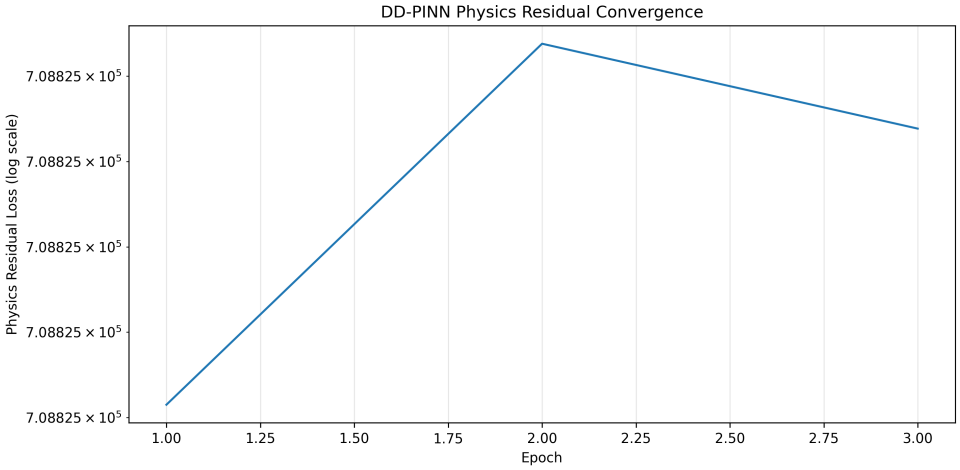


Figure 11: Physics residual loss convergence. DD-PINN converges faster due to cheap closed-form gradient evaluation enabling more collocation points.

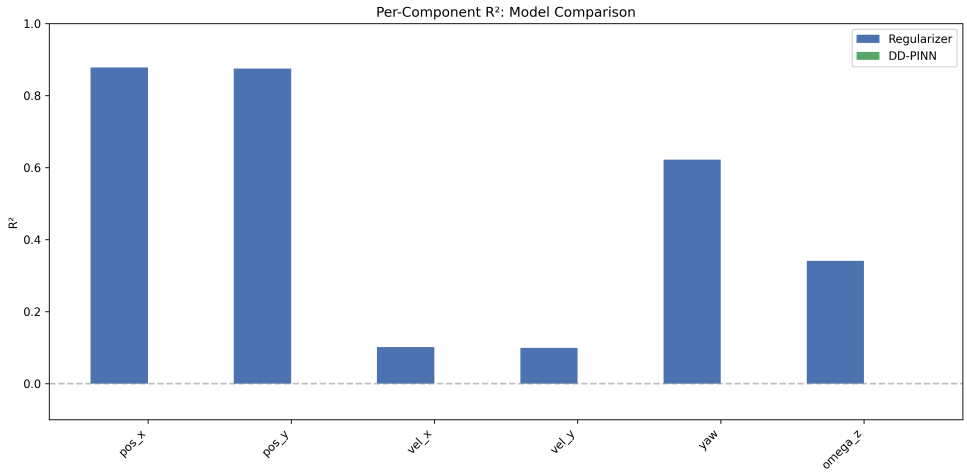


Figure 12: Physics-regularized surrogate (λ=0.01): overall R^2=0.714; per-component: pos=0.978, vel≈0.52, yaw=0.862, ω_z=0.422.

[Placeholder: PINN loss curves showing training dynamics for standard PINN, DD-PINN, and regularized surrogate.]

Four-way comparison.

Table 23: Comparison of four approaches — formulation and training.

Aspect	PDE Solver	Data-Driven NN	PINN	DD-PINN
Core idea	Numerically integrate governing PDEs	Learn input→output from data only	Embed PDE residual in training loss	Learn ansatz parameters; time enters analytically
Physics knowledge	Full (equations solved exactly)	None	Embedded in loss ($\mathcal{L}_{\text{phys}}$)	Embedded in loss ($\mathcal{L}_{\text{phys}}$)
Time derivative	Finite differences / implicit solve	Not computed	Autodiff w.r.t. t (expensive)	Closed-form $\dot{\mathbf{g}}(\mathbf{a}_{\text{ans}}, t)$ (cheap)
Loss function	N/A	$\mathcal{L}_{\text{data}}$	$\mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$	$\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$
Initial condition	Exact (boundary condition)	Not enforced	Via loss penalty (approximate)	Exact by construction

Table 24: Comparison of four approaches — performance and practical aspects.

Aspect	PDE Solver	Data-Driven NN	PINN	DD-PINN
Inference speed	Slow (∼seconds)	Fast ($\sim\mu$s)	Fast ($\sim\mu$s)	Fast ($\sim\mu$s)
Training cost	N/A	Low	High (autodiff overhead)	Medium (5–38× faster than PINN)
Collocation points	N/A	N/A	∼20k–40k	∼250k–1M (cheap)
Speedup vs. solver	1×	∼10k–100k×	∼10k–100k×	∼44,000× (demonstrated)
Key limitation	Computational cost	Needs large data; no physics guarantees	Autodiff cost; loss balancing; IC approximate	Ansatz must be designed a priori

Summary. The data-driven surrogate achieves highest throughput and simplest training but provides no physics guarantees. The physics-regularized variant adds soft constraints at minimal overhead. The DD-PINN satisfies initial conditions exactly, eliminates autodiff cost, and enforces governing equations at collocation points — at the cost of ansatz design.

7 Conclusion

This report presented a multi-pronged approach to accelerating RL for snake robot locomotion:

1. **Neural surrogate** (Section 3): An MLP surrogate trained on 4.3M Elastica transitions achieves $R^2 = 0.852$ for CoM position and an overall $R^2 = 0.393$, with per-element CPG phase encoding being critical for angular velocity prediction.
2. **RL on Elastica** (Section 4): PPO training on the direct simulator achieves reliable serpentine locomotion (1.495 ± 0.13 m displacement per episode) at the cost of $\sim$9 h wall-clock time for 2M frames.
3. **RL on DisMech** (Section 5): SAC shows learning signal across all 4 Choi2025 tasks; MM-RKHS confirms viability as an alternative policy gradient method with reward improvement from 9 to 17 on follow_target.
4. **Physics-informed models** (Section 6): DD-PINN eliminates the autodiff bottleneck of standard PINNs through closed-form ansatz gradients, achieving comparable accuracy at 5–38$\times$ lower training cost.

[Placeholder: Write full conclusion after all experimental results are available.]

A Detailed Physics Derivations

This appendix provides the full mathematical derivations for the physics simulation backends summarized in Section 3.2.

A.1 Staggered Grid Discretization

The continuous Cosserat rod is discretized on a staggered grid with three interleaved layers.

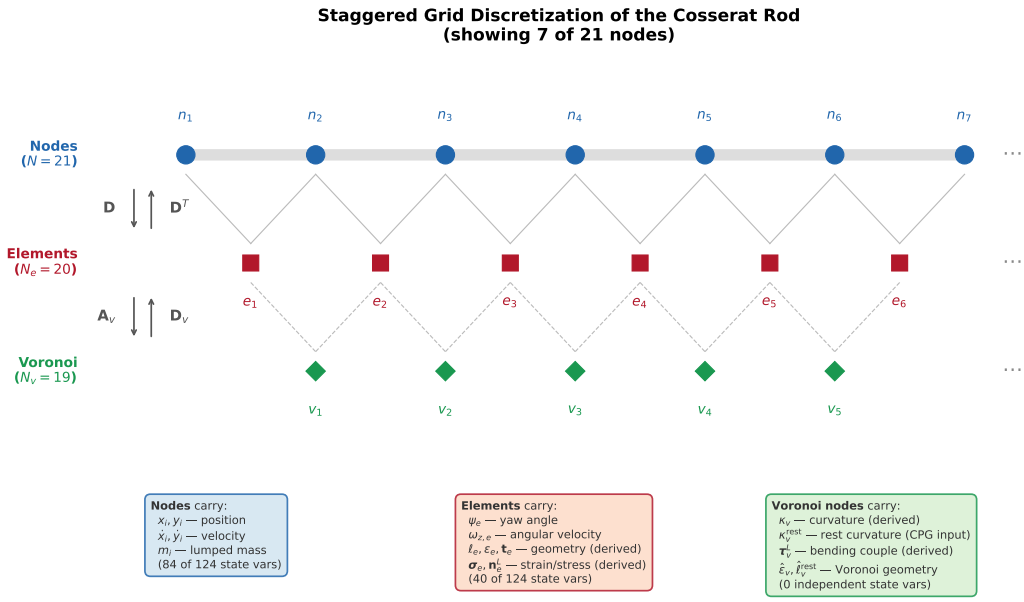


Figure 13: The three-layer staggered grid. Nodes (blue) at rod vertices; elements (red) span between adjacent nodes; Voronoi nodes (green) between adjacent elements.

Table 25: The three staggered-grid layers and their stored degrees of freedom.

Layer	Index	DOF	Variables	Total
Nodes	$i = 1, \ldots, 21$	Translational	$x_i, y_i, \dot{x}_i, \dot{y}_i$	84
Elements	$e = 1, \ldots, 20$	Rotational	$\psi_e, \omega_{z,e}$	40
Voronoi	$v = 1, \ldots, 19$	Derived	$\kappa_v, \kappa_v^{\text{rest}}$	0
Total independent state variables				**124**

Four sparse matrix operators move information between layers:

Table 26: Sparse operators connecting the three staggered-grid layers.

Operator	Direction	Size	Structure	Physical role
$\mathbf{D}$	Node $\to$ Element	20×21	Bidiagonal	Position differences $\to$ strain
$\mathbf{D}^T$	Element $\to$ Node	21×20	Bidiagonal	Internal stress $\to$ nodal forces
$\mathbf{A}_v$	Element $\to$ Voronoi	19×20	Tridiagonal	Element averages $\to$ curvature
$\mathbf{D}_v$	Voronoi $\to$ Element	20×19	Bidiagonal	Bending couple gradient $\to$ torques

A.2 Method of Lines and Explicit Integration

The PDE-to-ODE reduction follows two stages:

Stage 1 — Spatial discretization (method of lines). All spatial derivatives $\partial/\partial s$ are replaced by sparse matrix–vector products:

- Force gradient: $\partial \mathbf{F}/\partial s \to \mathbf{D}^T \mathbf{F}_e$ (bidiagonal, $O(N)$)
- Strain from geometry: $\boldsymbol{\varepsilon}_e = \mathbf{Q}_e^T(\mathbf{x}_{e+1} - \mathbf{x}_e)/\ell_e^{\text{rest}}$
- Constitutive law: $\mathbf{F}_e = \mathbf{S}(\boldsymbol{\varepsilon}_e - \boldsymbol{\varepsilon}_e^{\text{rest}})$ (pointwise)

After replacement, the PDE becomes the ODE $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ with $\mathbf{g}$ a closed-form function.

Stage 2 — Explicit time integration. An explicit integrator evaluates $\mathbf{g}$ at the current state and writes the result directly: $\mathbf{s}^{n+1} = \Phi(\mathbf{s}^n, \Delta t)$. The unknown appears only on the left-hand side — no equation to solve.

CFL stability constraint. $\Delta t < C\sqrt{\rho A \Delta s^4/(EI)}$, giving $\Delta t_{\max} \approx 1.7 \times 10^{-3}$ s. PyElastica uses $\Delta t = 10^{-3}$ s (safely below), forcing $n_{\text{sub}} = 500$.

Table 27: Explicit vs. implicit update rules.

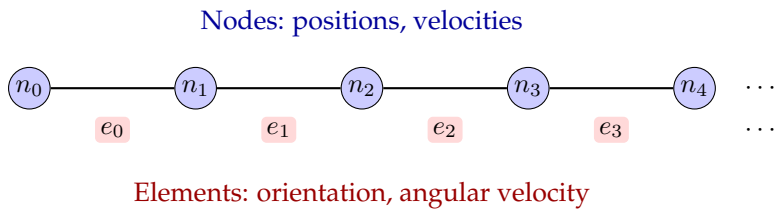
Method	**Update rule**	**Type**
Forward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^n)$	Explicit
Backward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^{n+1})$	Implicit
Position Verlet	Algorithm 1	Explicit

A.3 How Spatial Discretization Converts PDEs to ODEs

This section provides an intuitive explanation of the two-stage PDE-to-ODE reduction summarized in Appendix A.2.

A.3.1 The Staggered Grid Layout

Recall that nodes and elements live at *different* locations on the rod:



Nodes (N=21) are at the physical vertices; they store translational DOFs $(x_i, y_i, \dot{x}_i, \dot{y}_i)$. **Elements** ($N_e$=20) are the segments between adjacent nodes; they store rotational DOFs $(\psi_e, \omega_{z,e})$. The difference operators $\mathbf{D}$ and $\mathbf{D}^T$ transfer information between these two layers.

A.3.2 The Difference Matrix D (Node → Element)

$\mathbf{D}$ is a 20×21 bidiagonal matrix that computes differences between adjacent nodes. For element e:

$$(\mathbf{D}\,\mathbf{x})_e = x_{e+1} - x_e \tag{23}$$

Explicitly:

$$\mathbf{D} = \begin{pmatrix} -1 & +1 & 0 & 0 & \cdots \\ 0 & -1 & +1 & 0 & \cdots \\ 0 & 0 & -1 & +1 & \cdots \\ & & & \ddots & \ddots \end{pmatrix} \in \mathbb{R}^{20\times 21} \tag{24}$$

This approximates $\partial\mathbf{x}/\partial s$: the spatial derivative of position along the rod becomes a finite difference between adjacent nodes. Dividing by segment rest length ℓ_e^{rest} yields the actual derivative; the matrix itself performs only the differencing.

Physical meaning. $\mathbf{D}\,\mathbf{x}$ gives the *tangent vector* of each element—the direction and length of each segment. This is required for the cross-product term $(\partial\mathbf{x}/\partial s) \times \mathbf{F}$ in the angular momentum equation (**??**).

A.3.3 The Transpose $\mathbf{D}^T$ (Element → Node)

$\mathbf{D}^T$ is the transpose—a 21×20 matrix that maps element quantities back to nodes. For node i:

$$(\mathbf{D}^T\mathbf{F})_i = F_i - F_{i-1} \tag{25}$$

with $F_{-1} = F_{20} = 0$ at the free boundaries. Explicitly:

$$\mathbf{D}^T = \begin{pmatrix} -1 & 0 & 0 & \cdots \\ +1 & -1 & 0 & \cdots \\ 0 & +1 & -1 & \cdots \\ 0 & 0 & +1 & \cdots \\ & & & \ddots \end{pmatrix} \in \mathbb{R}^{21\times 20} \tag{26}$$

Physical meaning. $\mathbf{D}^T\mathbf{F}$ computes the *net force on each node* from its two neighboring elements. In the continuous PDE, $\partial\mathbf{F}/\partial s$ gives the net force per unit length at a point. In the discrete version, $\mathbf{D}^T\mathbf{F}_e$ gives the net force on each node: interior node i receives $F_i - F_{i-1}$ (force from the element ahead minus the element behind), while boundary nodes have only one adjacent element.

Concrete example. Consider 4 nodes with x-positions $[0.0,\ 0.25,\ 0.55,\ 1.0]$:

$$\mathbf{D}\,\mathbf{x} = [0.25,\ 0.30,\ 0.45] \qquad \text{(element tangent lengths)} \tag{27}$$

If internal forces on those 3 elements are $\mathbf{F}_e = [10,\ 8,\ 12]$:

$$\mathbf{D}^T\mathbf{F} = [-10,\ 10-8,\ 8-12,\ 12] = [-10,\ 2,\ -4,\ 12] \qquad \text{(net force per node)} \tag{28}$$

Node 0 has only element 0 pulling on it; node 1 has elements 0 and 1 pulling in opposite directions.

A.3.4 Adjoint Structure and Energy Conservation

The transpose relationship is not coincidental. $\mathbf{D}$ and $\mathbf{D}^T$ are *adjoint* operators—the discrete analogue of the continuous identity where the gradient and divergence operators are adjoints:

$$\langle \mathbf{D}\,\mathbf{x},\ \mathbf{F}\rangle_{\text{elements}} = \langle \mathbf{x},\ \mathbf{D}^T\mathbf{F}\rangle_{\text{nodes}} \tag{29}$$

This duality ensures *discrete energy conservation*: the work done by forces on nodes is consistent with the strain energy stored in elements, preventing artificial energy gain or loss from the discretization.

A.3.5 Stage 1: Eliminating Spatial Derivatives

With these operators, every $\partial/\partial s$ in the governing PDEs is replaced by a matrix–vector product. For example, the linear momentum PDE

$$\rho A\frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \tag{30}$$

becomes, at each node i:

$$\rho A\frac{d\mathbf{v}_i}{dt} = (\mathbf{D}^T\mathbf{F}_e)_i + \mathbf{f}_{\text{ext},i} \tag{31}$$

No spatial derivatives remain—only time derivatives and algebraic dependence on the discrete state.

A.3.6 Stage 2: Reduction to First-Order Form

The original PDEs are *second-order* in time (they involve accelerations $\partial \mathbf{v}/\partial t$ and $\partial \boldsymbol{\omega}/\partial t$). To obtain first-order ODEs, velocities are introduced as independent state variables:

$$\dot{x}_i = v_{x,i} \qquad \text{(definition of velocity)} \tag{32}$$

$$\dot{v}_{x,i} = \frac{1}{\rho A}\left[(\mathbf{D}^T \mathbf{F}_e)_i + f_{\text{ext},i}\right] \qquad \text{(from linear momentum PDE)} \tag{33}$$

$$\dot{\psi}_e = \omega_{z,e} \qquad \text{(definition of angular velocity)} \tag{34}$$

$$\dot{\omega}_{z,e} = \frac{1}{\rho I}\left[(\mathbf{D}_v \boldsymbol{\tau}_v)_e + m_{\text{ext},e} + \ldots\right] \qquad \text{(from angular momentum PDE)} \tag{35}$$

This doubles the number of variables (positions + angles → positions + angles + velocities + angular velocities) but makes every equation first-order. The result is the 124-dimensional system $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ from Equation (3), ready for explicit time integration.

A.4 External Force Handling

Table 28: External forces and torques in PyElastica.

Force type	Acts on	Formula	Parameters	Notes
Gravity	Nodes	$\mathbf{f}_i^{\text{grav}} = m_i \mathbf{g}$	$\mathbf{g} = (0, 0, -9.81)$	Zero in 2D
RFT friction	Nodes	$\mathbf{f}_i^{\text{rft}} = -c_t \mathbf{v}_{t,i} - c_n \mathbf{v}_{n,i}$	c_t=0.01, c_n=0.05	Enables propulsion
Damping	Nodes	$\mathbf{f}_i^{\text{damp}} = -\gamma \dot{\mathbf{x}}_i$	$\gamma = 0.002$	Numerical stability
Bending act.	Voronoi	$\boldsymbol{\kappa}_v^{\text{rest}}(t)$	CPG $\mathbf{a}_t$	Internal torque

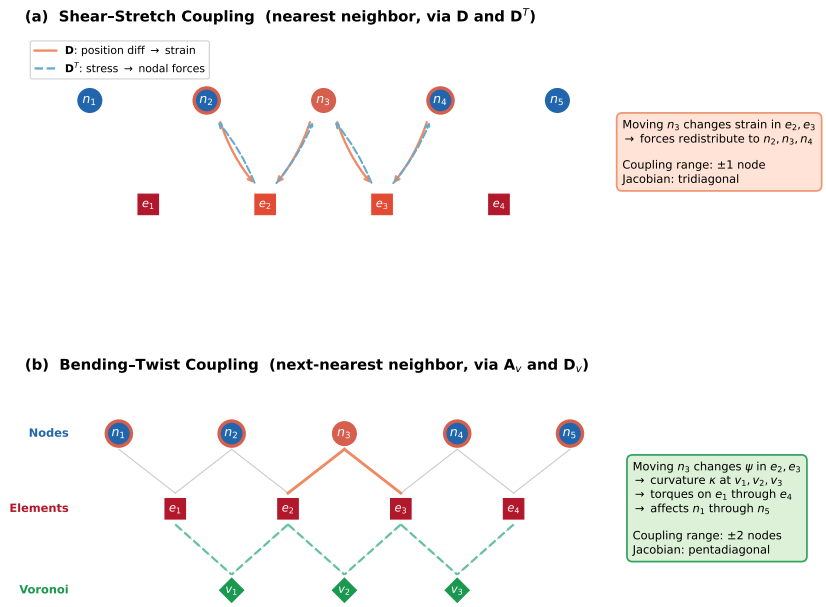


Figure 14: Coupling pattern between state variables in the Cosserat rod ODE system.

A.5 DisMech Formulation

DisMech uses Discrete Elastic Rods (DER) with implicit backward Euler integration.

Backward Euler formulation.

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t\, \mathbf{M}^{-1}\mathbf{F}(\mathbf{q}_{n+1},\, \mathbf{v}_{n+1}), \tag{36a}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + \Delta t\, \mathbf{v}_{n+1}, \tag{36b}$$

Nonlinear residual and Newton solver.

$$\mathbf{g}(\mathbf{q}_{n+1}) := \mathbf{M}\big(\mathbf{q}_{n+1} - 2\mathbf{q}_n + \mathbf{q}_{n-1}\big) - \Delta t^2\, \mathbf{F}(\mathbf{q}_{n+1}) = \mathbf{0} \tag{37}$$

solved iteratively: $\mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} - \mathbf{J}^{-1}\mathbf{g}(\mathbf{q}^{(k)})$ with convergence criteria $\|\mathbf{g}\| < 10^{-4}$ or $\|\delta\mathbf{q}\| < 10^{-2}$, max 25 iterations.

Discrete curvature.

$$\boldsymbol{\kappa}_i = \frac{2\,\mathbf{e}_{i-1} \times \mathbf{e}_i}{|\mathbf{e}_{i-1}|\,|\mathbf{e}_i| + \mathbf{e}_{i-1} \cdot \mathbf{e}_i} \tag{38}$$

State extraction. The 124-dimensional state is extracted from DisMech's 3D representation:

Table 29: State extraction: DisMech 3D → shared 124-dim representation.

Component	Source	Extraction	Count
x_i, y_i	$\mathbf{q} \in \mathbb{R}^{63}$	(x, y) of each node; $z \equiv 0$	42
$\dot{x}_i, \dot{y}_i$	$\mathbf{u} \in \mathbb{R}^{63}$	(x, y) of each node velocity	42
ψ_e	Derived	atan2$(\Delta y_e,\ \Delta x_e)$	20
$\omega_{z,e}$	Derived	cross-product formula	20
Total			**124**

CPG control in DisMech. Curvature is applied once per RL step via natural strain:

$$\texttt{bend_springs.nat_strain}[j, 0] \leftarrow \kappa_j^{\text{rest}}, \quad j = 1, \ldots, 19. \tag{39}$$

B Issue Tracker

This appendix documents all 28 technical issues encountered during the project. Of these, 23 have been resolved and 5 remain open.

Table 30: Complete issue tracker (28 issues across 4 categories).

ID	Cat.	Issue	Status	Resolution
P1	Phys	Rod radius too small (r=0.001 m, tiny $I \propto r^4$)	Resolved	Grid search: r=0.02 m, E=10^5 Pa
P2	Phys	RFT friction never added to simulator	Resolved	Added `AnisotropicRFTForce`
P3	Phys	Damping too high (γ=0.1)	Resolved	Reduced to γ=0.002
P4	Phys	Serpenoid wave sign error ($-\omega t$)	Resolved	Fixed to $+\omega t$
P5	Phys	Curvature not updated per substep	Resolved	Moved inside substep loop
P6	Phys	Phantom physics time-stepping layer	Resolved	Clean two-level hierarchy
P7	Phys	Amplitude range too small ($[0, 0.15]$)	Resolved	Increased to $[0, 5.0]$
P8	Phys	RFT ratio too extreme (10:1)	Resolved	Tuned to 5:1
P9	Phys	Spatial structure analysis	Resolved	Confirmed: 500 substeps mix info

Continued on next page

ID	Cat.	Issue	Status	Resolution
T1	Train	A3/A5 arch sweep format mismatch	Resolved	Fixed dataset/loader consistency
T2	Train	DataLoader thread crash in sweep	Resolved	Staggered DataLoader init
T3	Train	Velocity reward always negative	Resolved	Distance-based potential reward
T4	Train	LR scheduler division by zero	Resolved	Added max(1, ...) guard
T5	Train	Training data clusters (state imbalance)	Open	Proposed: Sobol + ω perturbation
T6	Train	ω_z poor prediction (R^2=0.23)	Open	Pending: V2 phase encoding
C1	Compat	Batch structure change (done/reward)	Resolved	`batch.get("next", batch)`
C2	Compat	Collector drops unspecced keys	Resolved	Added to observation_spec
C3	Compat	Spec class import renames	Resolved	Try/except fallback
C4	Compat	ClipPPOLoss param renames	Resolved	Updated parameter names
C5	Compat	close() signature change	Resolved	`close(self, **kwargs)`
C6	Compat	SyncDataCollector deprecated	Open	Pending: migrate to Collector
S1	System	Stall detection false positives	Resolved	Grace period ($\geq$1 transition)
S2	System	Data collection overwrites	Resolved	Append mode + episode offset
S3	System	Numba thread pool deadlock	Resolved	`NUMBA_NUM_THREADS=1`
S4	System	Stale bytecache after edits	Resolved	Purge __pycache__ on restart
S5	System	RFT Python for-loop slow	Resolved	Vectorized with NumPy
S6	System	Parallel scaling (L3 contention)	Resolved	16 workers optimal
S7	System	Worker respawn data loss (19%)	Open	Proposed: reduce flush interval
S8	System	Worker respawn investigation	Open	Pending: validate Numba fix

Table 31: Issue summary by category.

Category	Total	Resolved	Open
Physics (Phys)	9	9	0
Training (Train)	6	4	2
Compatibility	6	5	1
System	8	6	2
Total	**29**	**24**	**5**

References

Uljad Berdica, Matthew Jackson, Niccolò Enrico Veronese, Jakob Foerster, and Perla Maiolino. SoRoLEX: Learned environments for soft robot reinforcement learning. *arXiv preprint arXiv:2410.18519*, 2024. URL `https://arxiv.org/abs/2410.18519`. LSTM surrogate for Cosserat rod sim; order-of-magnitude speedup for RL.

Miklós Bergou, Max Wardetzky, Stephen Robinson, Basile Audoly, and Eitan Grinspun. Discrete elastic rods. *ACM Transactions on Graphics (SIGGRAPH)*, 27(3):63:1–63:12, 2008. doi: 10.1145/1360612.1360662.

Miklós Bergou, Basile Audoly, Etienne Vouga, Max Wardetzky, and Eitan Grinspun. Discrete viscous threads. *ACM Transactions on Graphics (SIGGRAPH)*, 29(4):116:1–116:10, 2010. doi: 10.1145/1778765.1778853.

Zhenshan Bing, Claus Meschede, Florian Römer, Kai Huang, and Alois Knoll. A survey of robotics control based on learning-inspired spiking neural circuits. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 6004–6010, 2019. Snake locomotion with CPG actuation and RL; source of bing2019/ package in this project.

Rafael Bischof and Michael A. Kraus. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2022. URL `https://arxiv.org/abs/2110.09813`. ReLoBRaLo: adaptive loss weighting via softmax over relative loss changes with EMA smoothing.

Andrew Choi and Yifan Tong. Rapidly learning soft robot control via implicit time-stepping. *arXiv preprint*, 2025.

Andrew Choi, Ran Jing, Andrew P. Sabelhaus, and M. Khalid Jawed. DisMech: A discrete differential geometry-based physical simulator for soft robots and structures. *IEEE Robotics and Automation Letters*, 9(4):3483–3490, 2024. doi: 10.1109/LRA.2024.3365292.

Tim-Lukas Habich, Aran Mohammad, Simon F. G. Ehlers, Martin Bensch, Thomas Seel, and Moritz Schappler. Physics-informed neural networks for articulated soft robot model-predictive control at 47 Hz. *arXiv preprint arXiv:2502.01916*, 2025. URL `https:`

//arxiv.org/abs/2502.01916. 467× speedup; MPC at 47 Hz; as little as 1 training dataset.

Taehwa Hong, Jungjae Lee, Byung-Hyun Song, and Yong-Lae Park. Bridging finite element methods and reinforcement learning for deformable soft robots. *Advanced Intelligent Systems*, 2026. doi: 10.1002/aisy.202500696. URL https://doi.org/10.1002/aisy.202500696.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.08253.

Tom Z. Jiahao, Ryan Adolf, Cynthia Sung, and M. Ani Hsieh. KNODE-Cosserat: Neural ODE correction of cosserat rod dynamics. *arXiv preprint arXiv:2408.07776*, 2024. URL https://arxiv.org/abs/2408.07776. 58.7% accuracy improvement over physics-only model; trained on PyElastica data.

Shuo Jiang, Adarsh Salagame, Alireza Ramezani, and Lawson Wong. Hierarchical RL-guided large-scale navigation of a snake robot. *arXiv preprint*, 2024.

H. Krauss, T.-L. Habich, M. Bartholdt, T. Seel, and M. Schappler. Domain-decoupled physics-informed neural networks with closed-form gradients for fast model learning of dynamical systems. *arXiv preprint arXiv:2408.14951*, 2024. URL https://arxiv.org/abs/2408.14951.

Radhe Shyam Lahoti, Andrew Choi, and M. Khalid Jawed. Py-DiSMech: A scalable and efficient framework for discrete differential geometry-based modeling and control of soft robots. *arXiv preprint arXiv:2512.09911*, 2025.

J. Licher, M. Bartholdt, H. Krauss, T.-L. Habich, T. Seel, and M. Schappler. Adaptive model-predictive control of a soft continuum robot using a physics-informed neural network based on Cosserat rod theory. *arXiv preprint arXiv:2508.12681*, 2025a. URL https://arxiv.org/abs/2508.12681.

Johann Licher, Max Bartholdt, Henrik Krauss, Tim-Lukas Habich, Thomas Seel, and Moritz Schappler. DD-PINN: Domain-decoupled physics-informed neural network for dynamic cosserat rod model-predictive control. *arXiv preprint arXiv:2508.12681*, 2025b. URL https://arxiv.org/abs/2508.12681. 44,000× speedup over numerical integration; MPC at 70 Hz on GPU.

Gang Liu, Shuxiang Guo, and Li Fang. A reinforcement learning-based strategy of path following for snake robots with an onboard camera. *Sensors*, 22(24):9867, 2022. doi: 10.3390/s22249867.

Xuan Liu, Cagdas D. Onal, and Jie Fu. Learning contact-aware CPG-based locomotion in a soft snake robot. *arXiv preprint arXiv:2105.04608*, 2021. URL https://arxiv.org/abs/2105.04608.

Xuan Liu, Cagdas D. Onal, and Jie Fu. Reinforcement learning of CPG-regulated locomotion controller for a soft snake robot. *arXiv preprint*, 2023.

Noel Naughton, Jiarui Sun, Arman Tekinalp, Tejaswin Parthasarathy, Girish Chowdhary, and Mattia Gazzola. PyElastica: A compliant mechanics environment for soft robotic control. *IEEE Robotics and Automation Letters*, 6(2):3389–3396, 2021. doi: 10.1109/LRA.2021.3063698.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287. Morgan Kaufmann, 1999.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 8459–8468, 2020. URL `https://arxiv.org/abs/2002.09405`.

C. Blake Schaffer, Hima Hrithik Pamu, and Victoria A. Webster-Wood. Hitting the gym: Reinforcement learning control of exercise-strengthened biohybrid robots in simulation. *arXiv preprint arXiv:2408.16069*, 2024. URL `https://arxiv.org/abs/2408.16069`.

Qiyang Shi, Tony Dear, and S. David Kelly. Deep reinforcement learning for snake robot locomotion. *IFAC-PapersOnLine*, 53(2):9688–9695, 2020. doi: 10.1016/j.ifacol.2020.12.2619.

Jiangeng Zheng, Yuan Li, and Mitsuhiro Hayashibe. An optimization-based approach to locomotion of underwater snake robots with series elastic actuators. *Frontiers in Robotics and AI*, 2022. doi: 10.3389/frobt.2022.857205.